

Kamień Milowy II – Implementacja systemu

Aplikacja do zarządzania zadaniami

Bartłomiej Karkoszka

Rafał Grot

Filip Stępień

Przedmiot: Modelowanie i Analiza Systemów Informatycznych

Grupa: 1ID21A

Spis treści

1	Wprowadzenie	2
2	Cel i zakres systemu	2
2.1	Cel	2
2.2	Zakres	2
3	Aktorzy systemu	2
4	Wymagania funkcjonalne	3
5	Wymagania нефункционалне	3
6	Stos technologiczny	3
7	Słownik pojęć	4
8	Stan implementacji	4
8.1	Struktura repozytorium	4
8.2	Szczegóły implementacyjne	4
8.3	Lista endpointów REST	5
8.4	Schemat bazy danych	5
9	Diagram przypadków użycia	5
10	Diagram klas	6
11	Diagram komponentów	7
12	Diagram warstw systemu	9

13 Diagram sekwencji – utworzenie zadania	10
14 Diagram aktywności – zarządzanie zadaniem	11
15 Uruchamianie i weryfikacja	12
15.1 Wymagania wstępne	12
15.2 Kroki uruchomienia	12
15.3 Weryfikacja	13
16 Podsumowanie	13

1 Wprowadzenie

Niniejszy dokument stanowi **kontynuację i rozwinięcie** raportu z kamienia milowego I. Przedstawia on aktualny stan systemu po fazie implementacji, a także zawiera wszystkie sekcje analityczne z KM I – zaktualizowane zgodnie z rzeczywistą realizacją projektu. Dodatkowo wprowadzono dwa nowe diagramy wymagane w drugim etapie: diagram sekwencji oraz diagram aktywności.

W stosunku do KM I nastąpiła jedna istotna zmiana stosu technologicznego: zaplanowany Spring Boot (Java) po stronie backendu został zastąpiony przez **Go** (router **chi**, generator **sqlc**). Decyzja była motywowana chęcią wypróbowania języka Go w praktyce. Pozostałe elementy stosu (frontend React, Keycloak, PostgreSQL, Docker) pozostały bez zmian.

2 Cel i zakres systemu

2.1 Cel

Celem systemu jest umożliwienie użytkownikom efektywnego zarządzania zadaniami osobistymi i zespołowymi poprzez intuicyjny interfejs webowy. System pozwala na tworzenie, edycję, śledzenie postępu oraz organizację zadań z uwzględnieniem priorytetów i terminów realizacji.

2.2 Zakres

System obejmuje:

- rejestrację i uwierzytelnianie użytkowników (z wykorzystaniem serwisu Keycloak),
- zarządzanie zadaniami: tworzenie z tytułem, opisem, priorytetem i terminem realizacji, edycja oraz usuwanie,
- kategoryzację zadań według statusu (Do zrobienia, W trakcie, Zrobione) i priorytetu (Niski, Średni, Wysoki),
- widok listy zadań z możliwością filtrowania,
- panel główny (dashboard) z podsumowaniem statystyk zadań.

3 Aktorzy systemu

- **Użytkownik** – osoba korzystająca z systemu, która może się zalogować, zarejestrować oraz tworzyć, edytować, przeglądać i usuwać zadania.
- **Keycloak** – system zewnętrzny odpowiedzialny za uwierzytelnianie i autoryzację użytkowników (dostawca tokenów JWT).

4 Wymagania funkcjonalne

- Uwierzytelnianie i autoryzacja użytkowników poprzez serwis Keycloak (rejestracja, logowanie, wylogowanie).
- Zarządzanie zadaniami: tworzenie z tytułem, opisem, priorytetem i terminem realizacji, edycja oraz usuwanie.
- Wyświetlanie listy wszystkich zadań użytkownika.
- Zmiana statusu zadania.
- Przypisanie priorytetu do zadania.
- Wyświetlanie panelu dashboard z podsumowaniem liczby zadań w poszczególnych statusach.

Wszystkie powyższe wymagania zostały zaimplementowane w ramach drugiego kamienia milowego (szczegóły w sekcji 8).

5 Wymagania niefunkcjonalne

- Interfejs użytkownika musi być responsywny (desktop, tablet, mobile).
- Uwierzytelnianie musi być realizowane przez zewnętrzny serwer Keycloak – hasła użytkowników nie są przechowywane bezpośrednio w systemie.
- System powinien być łatwo wdrażalny za pomocą kontenerów Docker.
- Kod źródłowy musi być wersjonowany w repozytorium Git.

6 Stos technologiczny

- **Frontend:** React 19, TypeScript, Vite 8, Tailwind CSS v4, shadcn/ui, React Router v7, Redux Toolkit, react-redux.
- **Backend:** Go (router `chi`, generator zapytań `sqlc`, migracje `goose`, logger `slog`, dokumentacja `swaggo`).
- **Baza danych:** PostgreSQL 16.
- **Uwierzytelnianie:** Keycloak 26.2 (OAuth2 / OpenID Connect, PKCE, JWKS).
- **Konteneryzacja:** Docker, Docker Compose.
- **Środowisko deweloperskie:** Nix flake (`flake.nix`) oraz `Taskfile.yml`.
- **Kontrola wersji:** Git.

7 Słownik pojęć

- **Zadanie (Task)** – główna encja systemu, z którą użytkownik prowadzi interakcje; posiada tytuł, opis, status, priorytet i termin realizacji.
- **Status** – stan realizacji zadania: Do zrobienia, W trakcie, Zrobione.
- **Priorytet** – poziom ważności zadania: Niski, Średni, Wysoki.
- **Dashboard** – panel główny z podsumowaniem statystyk zadań użytkownika.
- **JWT (JSON Web Token)** – token używany do przekazywania informacji o tożsamości i uprawnieniach użytkownika między frontendem a backendem.
- **JWKS (JSON Web Key Set)** – zbiór publicznych kluczy serwera Keycloak udostępniany pod ścieżką `/protocol/openid-connect/certs`; używany do weryfikacji podpisów JWT.
- **sqlc** – generator kodu Go produkujący statycznie typowane funkcje na podstawie zapytań SQL.
- **Migracja** – numerowany skrypt SQL stosowany przy uruchomieniu aplikacji w celu doprowadzenia schematu bazy do oczekiwanego stanu.

8 Stan implementacji

8.1 Struktura repozytorium

Projekt zorganizowany jest jako monorepo zawierające dwie aplikacje:

- `apps/task-api` – backend w Go (REST API),
- `apps/task-frontend` – frontend w React 19 (SPA),

oraz konfigurację infrastruktury (`docker-compose.yml`, `docker/keycloak/realm.json`, `flake.nix`).

8.2 Szczegóły implementacyjne

Poniżej zebrano elementy implementacyjne uzupełniające wymagania funkcjonalne:

- **AuthMiddleware (chi)** – weryfikuje JWT w oparciu o klucze publiczne pobierane z JWKS Keycloak; przy pierwszym żądaniu nowego użytkownika automatycznie tworzy wpis w tabeli `users` na podstawie pól `sub`, `name`, `email` z tokenu.
- **Walidacja DTO** – metody `Validate()` na strukturach żądania kontrolują obecność tytułu, dopuszczalne wartości statusu/priorytetu oraz format daty (YYYY-MM-DD).
- **Generowane zapytania (sqlc)** – wszystkie operacje na bazie korzystają ze statycznie typowanych funkcji wygenerowanych z plików `queries/*.sql`; agregacja dashboardu wykonywana jest jednym zapytaniem `GROUP BY status`.

- **Swagger UI** – automatycznie generowana specyfikacja OpenAPI dostępna pod `/swagger`, z konfiguracją OAuth2/PKCE wskazującą na realm Keycloak.
- **Frontend (Redux Toolkit)** – stan zadań trzymany w slice z `createEntityAdapter`, operacje wykonywane przez thunki (`fetchTasks`, `createTask`, `deleteTask`).

8.3 Lista endpointów REST

Wszystkie endpointy są zabezpieczone middleware `Protect` (wymagany nagłówek `Authorization: Bearer <JWT>`) i udostępnione pod prefiksem `/api` (konfigurowalnym przez zmienną `API_BASE_PATH`).

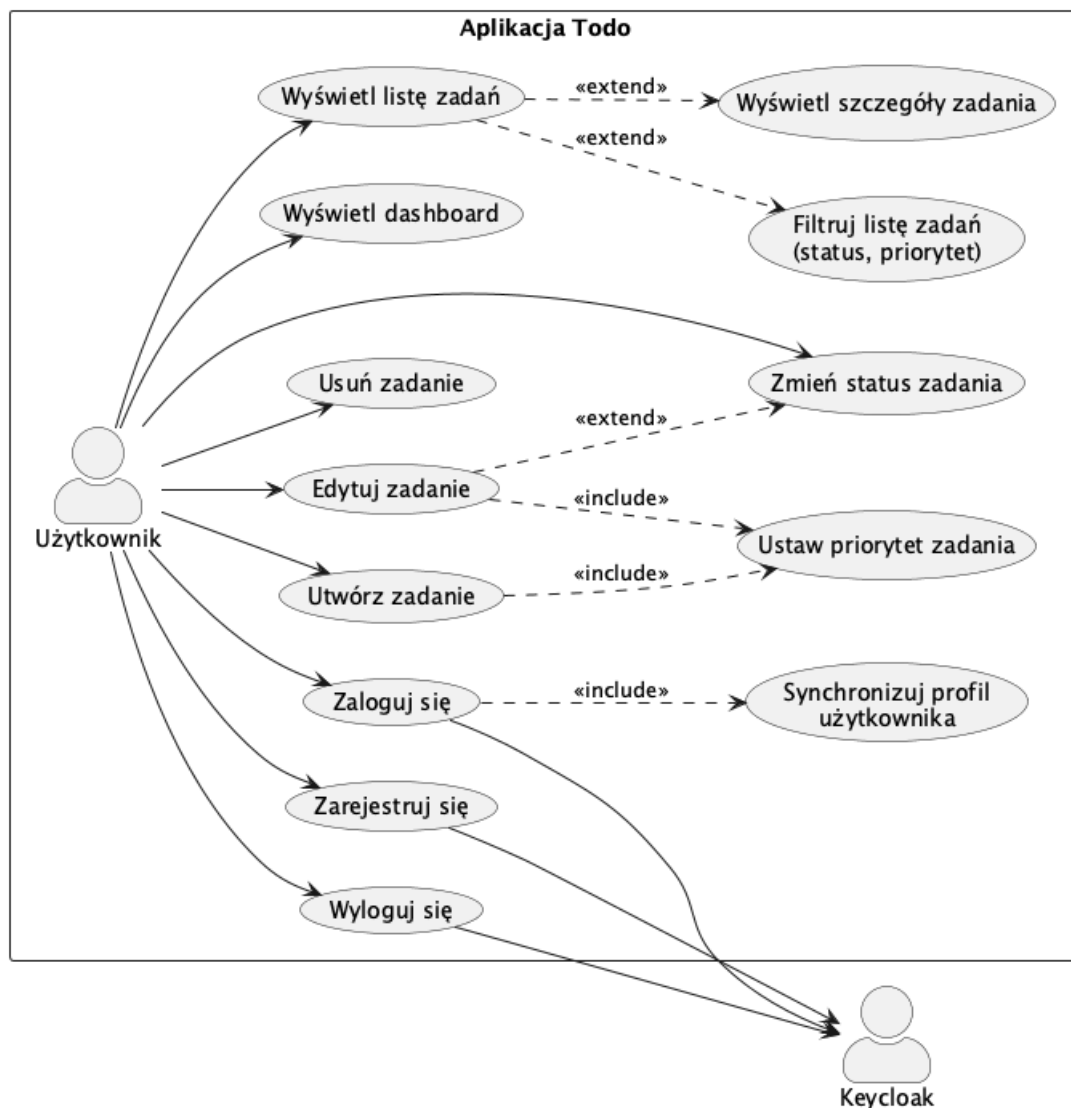
Metoda	Ścieżka	Opis	Kody
GET	<code>/health</code>	Stan aplikacji i bazy danych	200, 401, 500
GET	<code>/users/{userId}/tasks</code>	Lista zadań użytkownika z opcjonalnymi filtrami	200, 400, 401, 500
POST	<code>/users/{userId}/tasks</code>	Utworzenie nowego zadania	201, 400, 401, 500
GET	<code>/users/{userId}/tasks/{id}</code>	Szczegóły pojedynczego zadania	200, 400, 401, 404, 500
PUT	<code>/users/{userId}/tasks/{id}</code>	Aktualizacja zadania	200, 400, 401, 404, 500
DELETE	<code>/users/{userId}/tasks/{id}</code>	Usunięcie zadania	204, 400, 401, 404, 500
GET	<code>/users/{userId}/dashboard</code>	Statystyki zadań użytkownika	200, 400, 401, 500

8.4 Schemat bazy danych

Migracje (goose) tworzą tabele `users` oraz `tasks`, a także typy wyliczeniowe `task_status` (`todo`, `in_progress`, `done`) i `task_priority` (`low`, `medium`, `high`). Tabela `tasks` posiada klucz obcy `user_id` z kaskadowym usuwaniem oraz znaczniki czasowe `created_at`, `updated_at`.

9 Diagram przypadków użycia

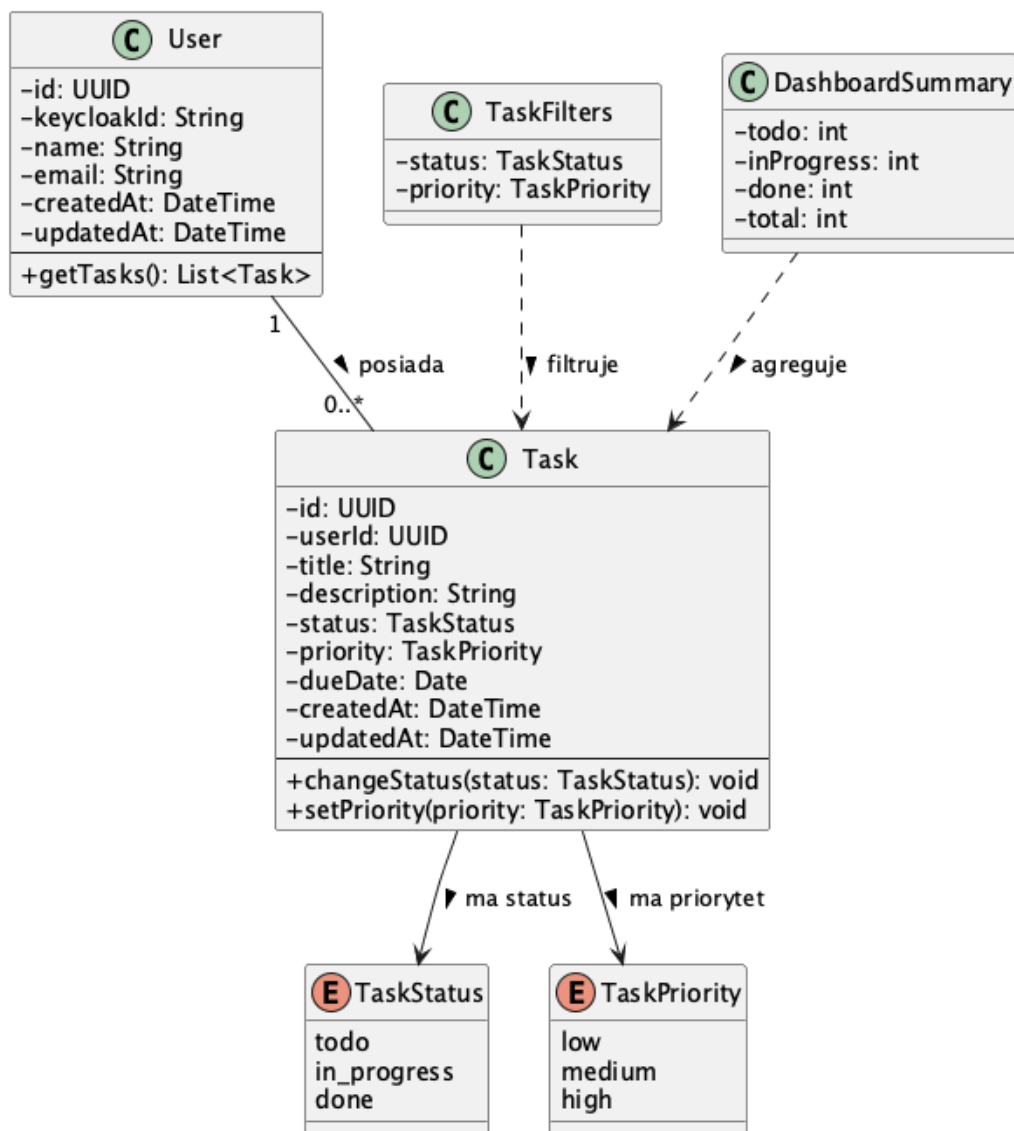
W stosunku do KM I diagram został otoczony granicą systemu *Aplikacja Todo*. *Użytkownik* – jako bezpośredni odbiorca aplikacji – został umieszczony wewnątrz granicy, natomiast *Keycloak* jako zewnętrzny dostawca uwierzytelniania pozostaje poza nią. Dodano przypadek *Synchronizuj profil użytkownika* (<<include>> w *Zaloguj się*) odpowiadający faktycznemu mechanizmowi tworzenia rekordu w tabeli `users` przy pierwszym żądaniu autoryzowanym.



Rysunek 1: Diagram przypadków użycia

10 Diagram klas

Klasa `User` została rozszerzona o pola `name`, `email`, `createdAt`, `updatedAt`, zgodnie z migracją `00002_add_user_profile_columns.sql`. Klasa `Task` otrzymała pole `userId` (klucz obcy). Wartości typów wyliczeniowych dostosowano do nazw rzeczywistych (`todo`, `in_progress`, `done`; `low`, `medium`, `high`). Dodano klasy pomocnicze `TaskFilters` oraz `DashboardSummary` odpowiadające DTO obecnym w kodzie.

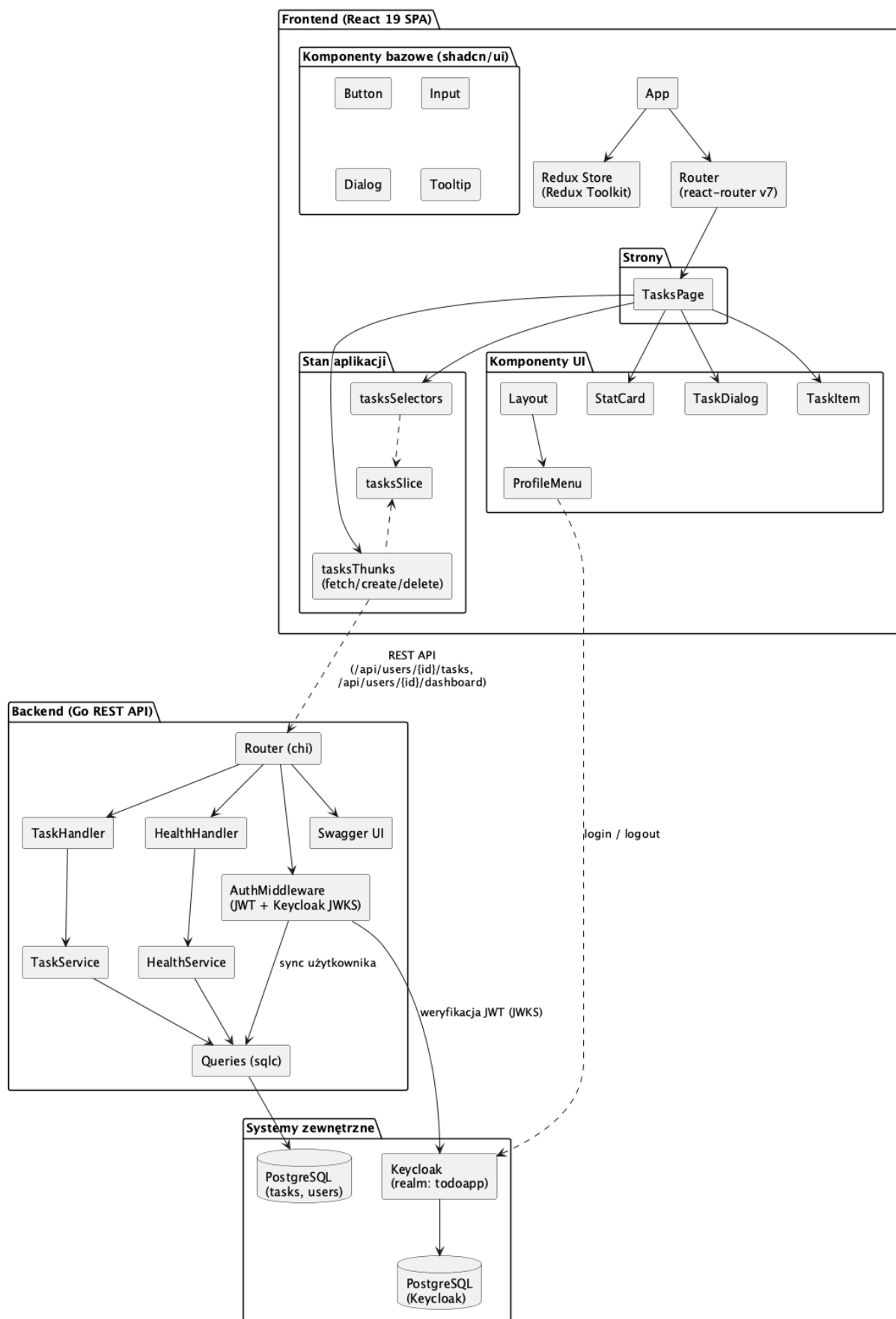


Rysunek 2: Diagram klas (model dziedziny + DTO)

Relacja: jeden użytkownik posiada wiele zadań (1 do 0..*).

11 Diagram komponentów

Diagram odzwierciedla rzeczywiste pakiety: po stronie frontendu wyodrębniono Redux Store (slice, thunki, selektory) oraz konkretne komponenty UI (`Layout`, `TaskDialog`, `TaskItem`, `StatCard`, `ProfileMenu`). Po stronie backendu przedstawiono router `chi`, `AuthMiddleware` z weryfikacją JWKS, handlery `TaskHandler` i `HealthHandler`, warstwę serwisów oraz wygenerowane przez `sqlc` `Queries`. Endpoint Swagger UI został wskazany jako osobny komponent.

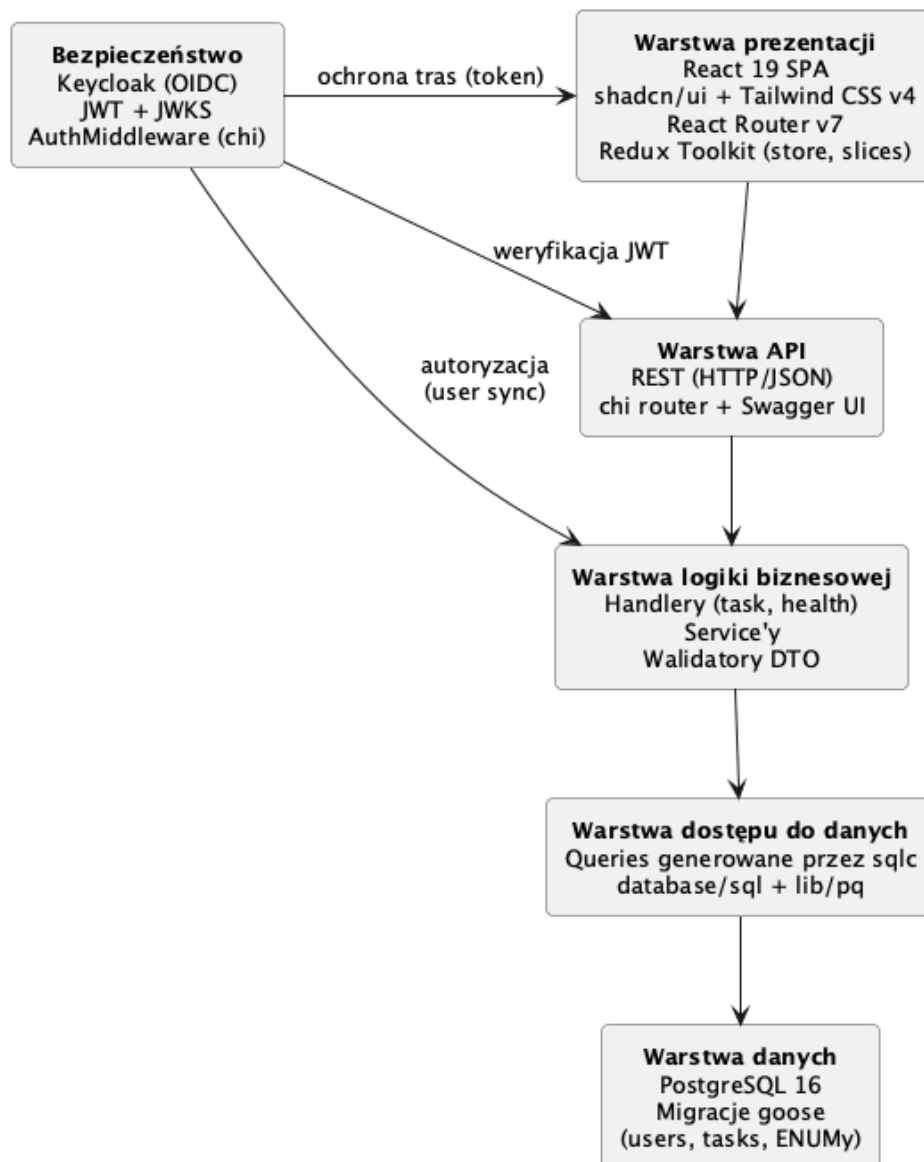


Rysunek 3: Diagram komponentów

Komunikacja między frontendem a backendem odbywa się przez REST API pod ścieżkami `/api/users/{id}/tasks` oraz `/api/users/{id}/dashboard`.

12 Diagram warstw systemu

Etykiety warstw zaktualizowano zgodnie z zaimplementowanymi technologiami: warstwa prezentacji opisuje stack React 19 + Redux Toolkit, warstwa API – chi i Swagger UI, warstwa logiki – handlers i serwisy, warstwa dostępu do danych – generowane przez `sqlc` zapytania, a warstwa bezpieczeństwa – middleware weryfikujący JWT.



Rysunek 4: Diagram warstw systemu

Architektura systemu oparta jest na modelu warstwowym:

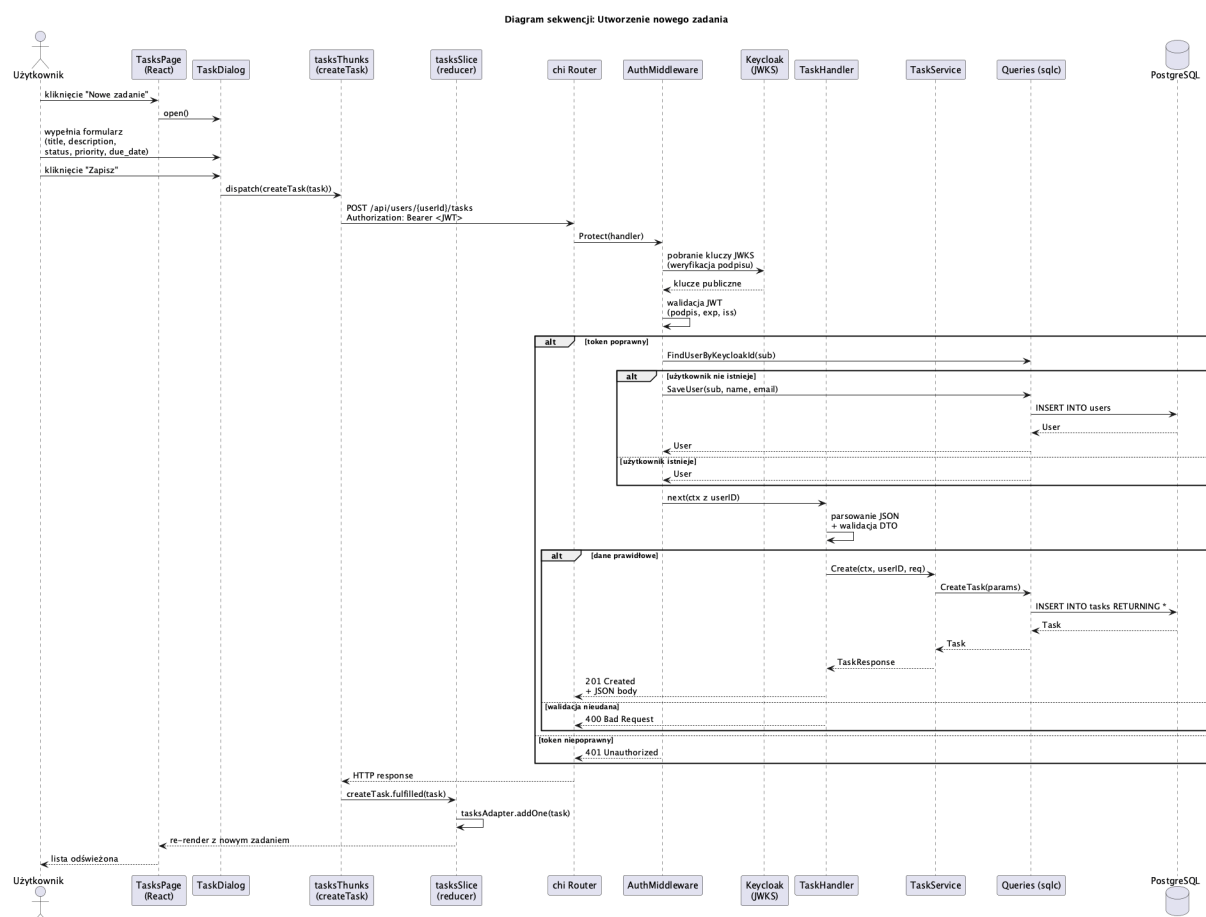
1. **Warstwa prezentacji** – aplikacja React SPA z Tailwind CSS, React Router oraz gotowymi komponentami shadcn/ui.

2. **Warstwa API** – komunikacja REST (HTTP/JSON) między frontendem a backendem.
3. **Warstwa logiki biznesowej** – handlers, serwisy oraz reguły walidacji.
4. **Warstwa dostępu do danych** – zapytania generowane przez sqlc.
5. **Warstwa danych** – relacyjna baza danych PostgreSQL.

Warstwa bezpieczeństwa (Keycloak, tokeny JWT) jest warstwą przekrojową, przenikającą zarówno warstwę prezentacji (ochrona tras), jak i warstwę logiki biznesowej (autoryzacja żądań).

13 Diagram sekwencji – utworzenie zadania

Wybrany scenariusz: użytkownik tworzy nowe zadanie. Scenariusz został wybrany, ponieważ ilustruje pełny przepływ przez wszystkie warstwy systemu – od interakcji w UI, przez akcję Redux i żądanie HTTP, weryfikację tokenu, synchronizację użytkownika, walidację DTO, persystencję w bazie, aż po aktualizację stanu po stronie klienta.



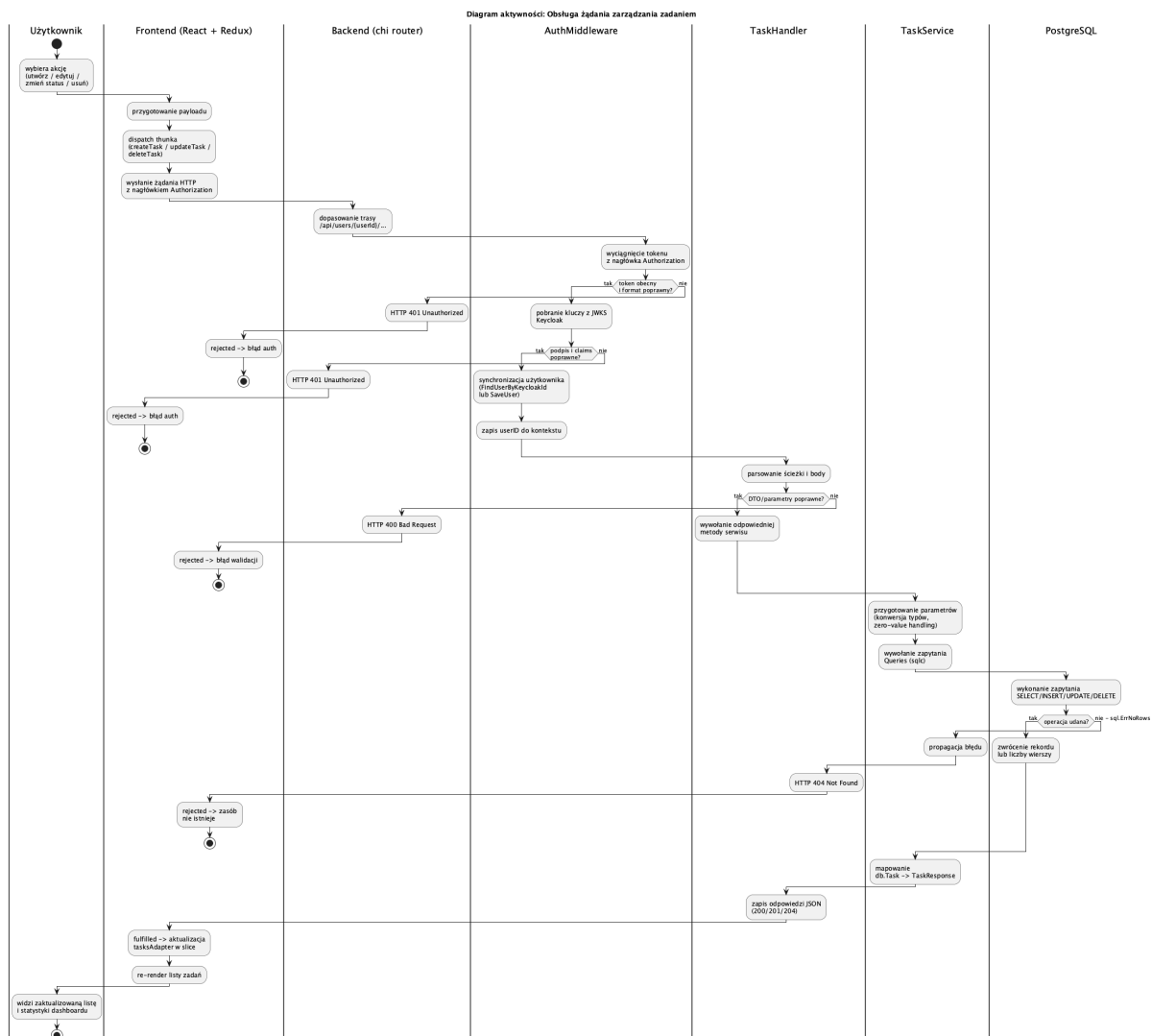
Rysunek 5: Diagram sekwencji: utworzenie nowego zadania

Opis kluczowych kroków.

1. Użytkownik otwiera dialog `TaskDialog` ze strony `TasksPage` i zatwierdza formularz.
2. Frontend wywołuje thunk `createTask`, który wysyła żądanie `POST /api/users/{userId}/tasks` wraz z tokenem JWT.
3. `AuthMiddleware` pobiera klucze z JWKS Keycloak i weryfikuje podpis tokenu.
4. Jeżeli użytkownik o podanym `sub` nie istnieje w bazie, jest tworzony (synchronizacja profilu); w przeciwnym wypadku rekord jest tylko odczytywany.
5. `TaskHandler` parsuje ciało żądania, uruchamia `Validate()` na DTO, a następnie wywołuje `TaskService.Create`.
6. Serwis przekazuje parametry do wygenerowanego zapytania `CreateTask`, które wykonuje `INSERT INTO tasks RETURNING *`.
7. Odpowiedź 201 `Created` z zserializowanym zasobem trafia do thunka, który wywołuje akcję `createTask.fulfilled` – reducer dodaje encję do `tasksAdapter`, co skutkuje ponownym renderowaniem listy.

14 Diagram aktywności – zarządzanie zadaniem

Wybrany proces: ogólna obsługa żądania zarządzania zadaniem (utworzenie, edycja, zmiana statusu, usunięcie). Diagram zawiera punkty decyzyjne dla wszystkich istotnych ścieżek błędów obecnych w implementacji: 401 (brak/niepoprawny token), 400 (błędne dane wejściowe) oraz 404 (`sql.ErrNoRows`).



Rysunek 6: Diagram aktywności: obsługa żądania zarządzania zadaniem

Tory (swimlanes). Diagram wyodrębnia odpowiedzialności komponentów: *Użytkownik*, *Frontend (React + Redux)*, *Backend (chi router)*, *AuthMiddleware*, *TaskHandler*, *TaskService* oraz *PostgreSQL*.

15 Uruchamianie i weryfikacja

15.1 Wymagania wstępne

Docker, Docker Compose, Go 1.22+, Node.js 20+ oraz task (Taskfile).

15.2 Kroki uruchomienia

1. Skopiowanie pliku `.env.example` do `.env` i uzupełnienie zmiennych.
2. Start usług infrastrukturalnych: `docker compose up -d postgres postgres-keycloak keycloak adminer`.

3. Wykonanie migracji bazy danych: `task migrate` (lub `goose`).
4. Uruchomienie API: `task api:dev` (`go run ./cmd`).
5. Uruchomienie frontendu: `npm run dev` w katalogu `apps/task-frontend`.

15.3 Weryfikacja

Po starcie systemu dostępne są:

- Swagger UI – `http://localhost:<API_PORT>/swagger/index.html`,
- panel Keycloak – `http://localhost:8080`,
- Adminer – `http://localhost:8081`,
- frontend – `http://localhost:5173`.

16 Podsumowanie

W ramach drugiego kamienia milowego dostarczono działającą wersję systemu obejmującą podstawowe scenariusze użycia: uwierzytelnianie przez Keycloak, zarządzanie zadaniami (CRUD), filtrowanie oraz dashboard. Dokumentacja projektowa została rozszerzona o dwa nowe diagramy (sekwencji oraz aktywności), a wcześniej przygotowane diagramy zaktualizowano zgodnie z rzeczywistą implementacją – w tym kluczową zmianą stosu backendowego (Go zamiast Spring Boot) oraz rozszerzonym modelem użytkownika.